Dictionaries and Tuples

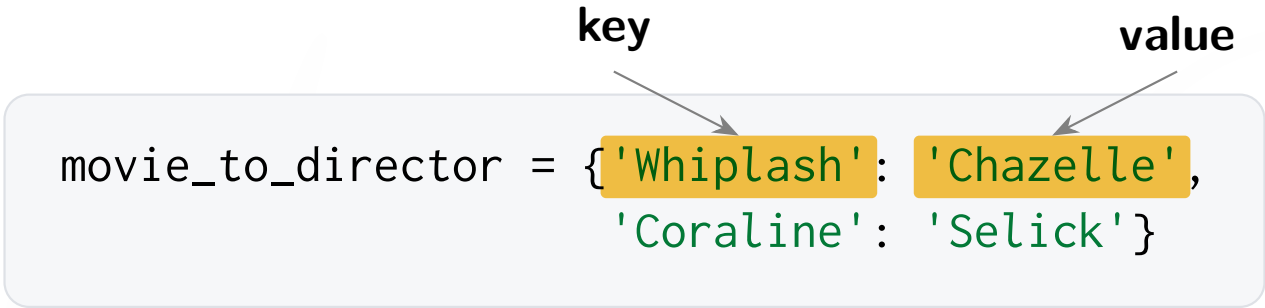
Introduction to Computer Programming

Chandra Gummaluru

chandra-gummaluru.github.io



A **dictionary** (`dict`) stores **key/value pairs**. We look a value up by its key, not by a position.



its type is `dict[str, str]`: keys first, then values

Square brackets with a key look up its value. A key that isn't in the `dict` raises a `KeyError`.

```
movie_to_director = {'Whiplash': 'Chazelle',
                     'Coraline': 'Selick'}
```

```
>>> movie_to_director['Whiplash']
'Chazelle'
>>> movie_to_director['Shrek']
KeyError: 'Shrek'
```

`in` checks the keys, not the values. `len` gives the number of key/value pairs.

```
movie_to_director = {'Whiplash': 'Chazelle',
                     'Coraline': 'Selick'}
```

```
>>> movie_to_director['Whiplash']
'Chazelle'
>>> movie_to_director['Shrek']
KeyError: 'Shrek'
>>> 'Coraline' in movie_to_director
True
>>> 'Selick' in movie_to_director
False
>>> len(movie_to_director)
2
```

Each key appears in a `dict` only once, and keys must be **immutable**. Values can be anything, including `lists`.

immutable	**mutable**
can be keys	can't be keys
`int` `float` `bool`	`list` `dict`
`str` `tuple`	

A `dict` is mutable: we can add, change and remove its pairs.

A **tuple** is an immutable sequence, written with round brackets. It works like a `list` except that it can't change, so it can be a `dict` key.

```
>>> movie = ('Shrek', 2001)
>>> movie[0]
'Shrek'
>>> movie + (90,)
('Shrek', 2001, 90)
>>> movie[1] = 2004
TypeError
```

```
>>> one = ('Shrek',)
>>> len(one)
1
>>> len(('Shrek'))
5
>>> ratings = {('Shrek', 2001): 8}
```

A `tuple` with one item needs a comma: `('Shrek')` is just the string `'Shrek'`. In this course, a `tuple` of strings has type `tuple[str]`.

Assigning to a key adds a new pair, or replaces the value if the key is already there.

```
>>> movie_to_rating = {'Shrek': 8}
>>> movie_to_rating['Coraline'] = 8
>>> movie_to_rating['Shrek'] = 9
>>> movie_to_rating
{'Shrek': 9, 'Coraline': 8}
```

adds a pair

replaces the value

A value that is a `list` can be changed with `list` methods.

```
>>> movie_to_actors = {'Shrek': ['Myers']}
>>> movie_to_actors['Shrek'].append('Murphy')
>>> movie_to_actors
{'Shrek': ['Myers', 'Murphy']}
```

A `for` loop over a `dict` goes through its **keys**.

```
total = 0
for item in item_to_price:
    total = total + item_to_price[item]
```

looping over a `dict`

```
counts = {}
for word in words:
    if word not in counts:
        counts[word] = 0
    counts[word] = counts[word] + 1
```

building a `dict`

To build a `dict`, start with `{}` and add a key the first time it's seen.

dicts and .csv files

When a `.csv` file has a header row, **`csv.DictReader`** gives each of the other rows as a `dict`. The keys are the column names from the header.

```
>>> import csv
>>> with open('watchlist.csv') as watch_file:
...     reader = csv.DictReader(watch_file)
...     for row in reader:
...         print(row['Title'], row['Rating'])
...
Whiplash 9
Shrek 8
Coraline 8
```

the first `row` is

`{'Title': 'Whiplash', 'Year': '2014', 'Genre': 'Drama', 'Rating': '9'}`

`row['Rating']` is clearer than `row[3]`; a constant `RATING` is clearer still.